



Blazor Essencial

Apresentando RenderFragment





Componentes complexos

Componente Pai

Números
Textos
Objetos

Parâmetros

Componente Filho

Cenário Simples

RenderFragment

Componente Pai

pedaço de
interface
(HTML + Razor)

Parâmetros

Componente Filho

Cenário Complexo

O **RenderFragment** é um **delegate** que representa um **bloco de interface renderizável**. Ele é um tipo usado para definir um parâmetro especial que permite ao usuário de um componente **injetar conteúdo arbitrário** (*texto, HTML, outros componentes*) dentro do componente.

Ele é como um "**pedaço de interface**" que você pode passar para um componente permitindo que o componente receba um bloco de marcação (**HTML + Razor**) fornecido por quem está usando o componente, além de dados e conteúdo diverso.

Temos duas formas de usar esse recurso:

RenderFragment → Representa um trecho de interface que o componente pode receber e renderizar, sem precisar de nenhum parâmetro extra.

RenderFragment<T> → Representa um trecho de interface, mas neste caso o componente passa um valor do tipo **T** para esse trecho, e ele pode usar esse valor durante a renderização.

@ Sintaxe do RenderFragment com ChildContent

RenderFragment

@ChildContent

```
@code {  
    [Parameter]  
    public RenderFragment? ChildContent { get; set; }  
}
```

Teste.razor

ChildContent só pode ser usado com **RenderFragment**, não pode ser usado Com **RenderFragment<T>**

O **ChildContent** é um nome especial reservado pelo Blazor.

ChildContent representa um **bloco de marcação** (HTML + Razor) que será passado por quem utilizar o componente.

Qualquer conteúdo colocado entre as **tags** do componente que usar o componente **Teste** será injetado no **ChildContent**(Ex: **<Teste>** Qualquer conteúdo **</Teste>**)

@ Exemplo de RenderFragment com ChildContent

Home.razor

```
...  
<Painel>  
  <h4>Minhas Tarefas</h4>  
  <ListaTarefas ListaDeTarefas="@tarefas" OnRemoverTarefa="RemoveTarefa" />  
</Painel>  
...
```

Painel.razor

```
<div class="card p-3 my-2 shadow-sm">  
  @ChildContent  
</div>  
@code {  
  [Parameter]  
  public RenderFragment? ChildContent { get; set; }  
}
```

Tudo que estiver entre as tags do componente será passado automaticamente para esse parâmetro **ChildContent**.



Sintaxe do RenderFragment usando um nome específico

RenderFragment

@NomeDoFragmento

```
@code {  
    [Parameter]  
    public RenderFragment? NomeDoFragmento { get; set; }  
}
```

NomeDoFragmento representa um **bloco de marcação (HTML + Razor)** que será passado por quem utilizar o componente.

RenderFragment<T>

@NomeDoFragmento

```
@code {  
    [Parameter]  
    public RenderFragment<T>? NomeDoFragmento { get; set; }  
}
```

T é o **tipo** que o componente fornecerá ao fragmento, e que poderá ser usado na renderização.



Exemplo de RenderFragment com nome específico

Aviso.razor

```
<div> @Conteudo1</div>
<div> @Conteudo2 </div>
@code {
    [Parameter]
    public RenderFragment? Conteudo1 { get; set; }

    [Parameter]
    public RenderFragment? Conteudo2 { get; set; }
}
```

ListaTarefas.razor

```
...
<Aviso>
    <Conteudo1>  </Conteudo1>
    <Conteudo2><span style="color:red">
        <h3>Ainda não cadastrou tarefas !!!</h3>
    </span>
    </Conteudo2>
</Aviso>
...
```

Imagem **duvida1.png** na pasta
Wwwroot/images

@ Exemplo de uso de RenderFragment<T>

Para criar um componente genérico podemos usar **RenderFragment<T>** junto com a diretiva **@typeparam**.

O **RenderFragment<T>** é um delegate que permite passar um fragmento de interface parametrizado por um tipo **T**.

Já o **@typeparam** é uma diretiva que transforma o componente em genérico.

- Esta **diretiva** serve para dizer que o componente vai trabalhar com um tipo genérico, mas esse tipo não é fixo.
- Quem vai decidir qual é o tipo usado (Ex: **Tarefa**, **Cliente**, **Produto**) é quem usar o componente.
- Pense nisso como um **espaço reservado para um tipo**, que só será preenchido no momento em que o componente for utilizado.



Criando uma lista genérica com RenderFragment<T> e @typeparam

ListaGenerica.razor

@typeparam TItem

// Corpo do componente genérico

@code {

[Parameter, EditorRequired]

public IEnumerable<TItem> **Itens** { get; set; } = Enumerable.Empty<TItem>();

[Parameter, EditorRequired]

public RenderFragment<TItem> **ItemTemplate** { get; set; } = default!;

}

@typeparam T é uma diretiva do Blazor usada para declarar que um componente é genérico.

Itens é propriedade que representa a coleção de dados que o componente Blazor vai exibir.

ItemTemplate é um parâmetro do tipo **RenderFragment<T>** que define o modelo de renderização para cada item individual na lista.

O **corpo do componente** é a parte visual do componente que no nosso exemplo vai ser extraída parcialmente do componente ListaTarefas.razor:



Criando uma lista genérica com `RenderFragment<T>` e `@typeparam`

Corpo do componente

```
@if (Itens == null || !Itens.Any())
{
    <Aviso>
        <Conteudo1></Conteudo1>
        <Conteudo2>
            <span style="color:red">
                <h3>Sem itens para exibir...</h3>
            </span>
        </Conteudo2>
    </Aviso>
}
else
{
    <ul class="list-group">
        @foreach (var item in Itens)
        {
            <li class="list-group-item">
                @ItemTemplate(item)
            </li>
        }
    </ul>
}
```

Verifico se existem itens a serem exibidos e mostro uma imagem e uma mensagem usando o componente Aviso

É aqui que o Blazor vai injetar o layout que o consumidor definiu.

O componente não decide o visual dos itens, ele apenas entrega cada item para o `RenderFragment<T>` e deixa que o consumidor escolha o que exibir.

@ Usando o componente ListaGenerica

Lista.razor

```
<h3>Minhas Tarefas</h3>

<ListaGenerica TItem="Tarefa" Itens="tarefas">
  <ItemTemplate Context="tarefa">
    <div>
      <strong>@tarefa.Descricao</strong> - @(tarefa.Concluida ? "✅" : "❌")
      <br />
      <small>Criada em: @tarefa.DataCriacao.ToShortDateString()</small>
    </div>
  </ItemTemplate>
</ListaGenerica>

@code {
  private List<Tarefa>? tarefas;

  protected override async Task OnInitializedAsync()
  {
    //obtem a url base
    var url = navManager.BaseUri + "Dados/tarefas.json";
    var resultado = await http.GetFromJsonAsync<List<Tarefa>>(url);
    tarefas = resultado ?? new List<Tarefa>();
  }
}
```

TItem="Tarefa" → informa que o tipo da lista é Tarefa.

- O parâmetro **Itens** será uma coleção de Tarefa.
- O **ItemTemplate** receberá um objeto do tipo Tarefa.

Context="tarefa" → define o nome da variável local que representa cada item da lista.

Fluxo de execução:

O Blazor lê o parâmetro Itens (lista de tarefas) no **ListaGenerica**.

Para cada item da lista:

Chama o **ItemTemplate**, passando o item como **Context** (aqui chamado de **tarefa**).

Renderiza o HTML definido dentro do **ItemTemplate**.